AI BASED PERSONAL FINANCE MANAGER

Results

Department of Computer Science and Engineering, TKR College of Engineering & Technology

Batch C11 (CSE-C) · Academic year [ACADEMIC YEAR]

Table 1: Project team

S.No.	Student name	Roll number
1	Mekala Kalyan	23K91A05F0
2	Rajanala Sai Hemanth	23K91A05K3
3	Middala Surya Teja Reddy	23K91A05F2
4	Manga Akash	23K91A05E5

Project guide: N Padmavathi

Chapter and section numbers quoted in the text (for example Chapter 8 or Section 6.2) refer to the full project report, 17.Final Document. Figures and tables in this document are numbered from 1.

1. RESULTS AND OUTPUT SCREENS

1.1 Experimental set-up

FINJARVIS has no large training corpus, so this chapter reports what the running application shows (Section 8.2), how its three analytical modules behave when measured (Sections 8.3 to 8.5), how the assistant, the import pipeline and the API perform (Sections 8.6 and 8.7) and how the results compare with the base paper (Section 8.8). Every table and chart is drawn from `experiments/metrics.json` and `experiments/api_latency.json`, written by `scripts/run_experiments.py` and `scripts/measure_api.py`.

1.1.1 The data

The categoriser is trained on 51 short example descriptions in 11 categories that are written into `ml/expense_classifier.py`; no public dataset was used. The application data are the fictional records of the demo account (`demo` / `Demo@1234`): 68 transactions over six months (May to October 2026), five budgets, three savings goals and four

investment holdings, built from the 34-row file `data/sample_transactions.csv` (dates shifted to the current month) and three earlier synthetic months (seed 411, drift of about 4 % a month). **All of these data are fictional**: they show that the mechanisms work, not how a real household spends.

1.1.2 The probe set

Because the application has no train and test split of its own, the team wrote a probe set of 198 descriptions before scoring (`scripts/probe_data.py`): 18 for each of the 11 categories, in three tiers of 6 per category. Tier A uses wording close to the seed list, tier B upper-case bank-statement strings that contain a seed keyword (for example `UPI-HOSPITAL-BILL`), and tier C merchants and phrases the model has never seen (for example `BigBasket delivery`, `Jio 599 plan`). The probe set was never used for training. It is **not an independent benchmark**: the same team wrote the seed list and the probe set.

1.1.3 Machine and software

All measurements were made on one machine: an Intel Core i5-12500 (6 cores, 12 threads) with 15.7 GB of memory, an Intel UHD 770 integrated GPU (not used), Windows 11 Pro and Python 3.11.9, with scikit-learn 1.9.1. API timings use the backend on `127.0.0.1`, a throw-away SQLite file with the demo data, one client and a warm server.

Table 2: Evidence gathered and the file it comes from.

Measurement	What was done	Stored in
Categoriser	Accuracy, macro scores, confusion matrix, tiers, keyword baseline, leave-one-out, confidence and correction experiments on 198 probe descriptions	metrics.json: classifier
Forecast	One-step-ahead rolling-origin backtest on the demo account; synthetic simulation of 2,000 users per setting	metrics.json: forecast
Health score	Sensitivity sweeps from a baseline profile; 5,000 random profiles	metrics.json: health
Assistant	26 team-written questions (20 answerable, 6 off-topic) asked of the rule layer	metrics.json: assistant
Import	1,000-row CSV with 84 injected bad rows	metrics.json: import
API	30 repetitions per endpoint (10 for reports and import preview)	api_latency.json
Tests	38 pytest tests and 49 smoke-test checks	pytest and scripts/smoke_test.py output

1.2 Output screens

The screens below were captured from the running application with Playwright at 1440 by 900 pixels, logged in as the demo account; none is a mock-up. Every amount on them

belongs to the fictional demo records (Section 8.1.1). The light-theme landing page, the sign-up page and the light-theme dashboard are in Appendix C.

1.2.1 Landing and login

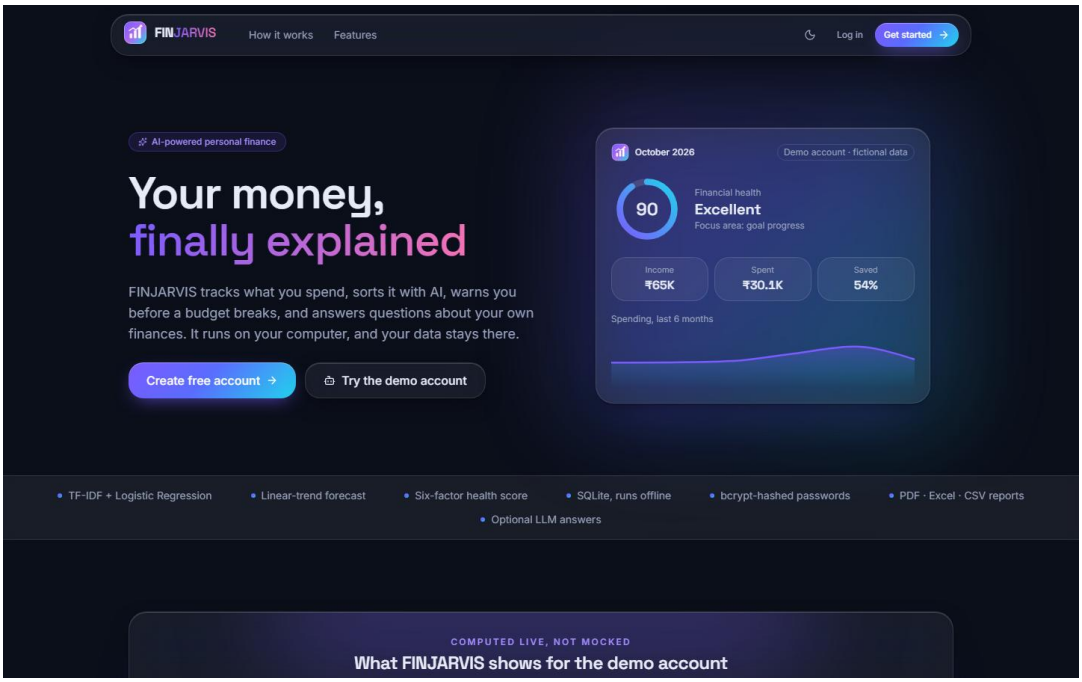


Figure 1: Landing page (dark theme): the hero card shows the demo account for October 2026 with health score 90 (Excellent), income INR 65K, spent INR 30.1K and 54 % saved; the strip below lists what the product does.

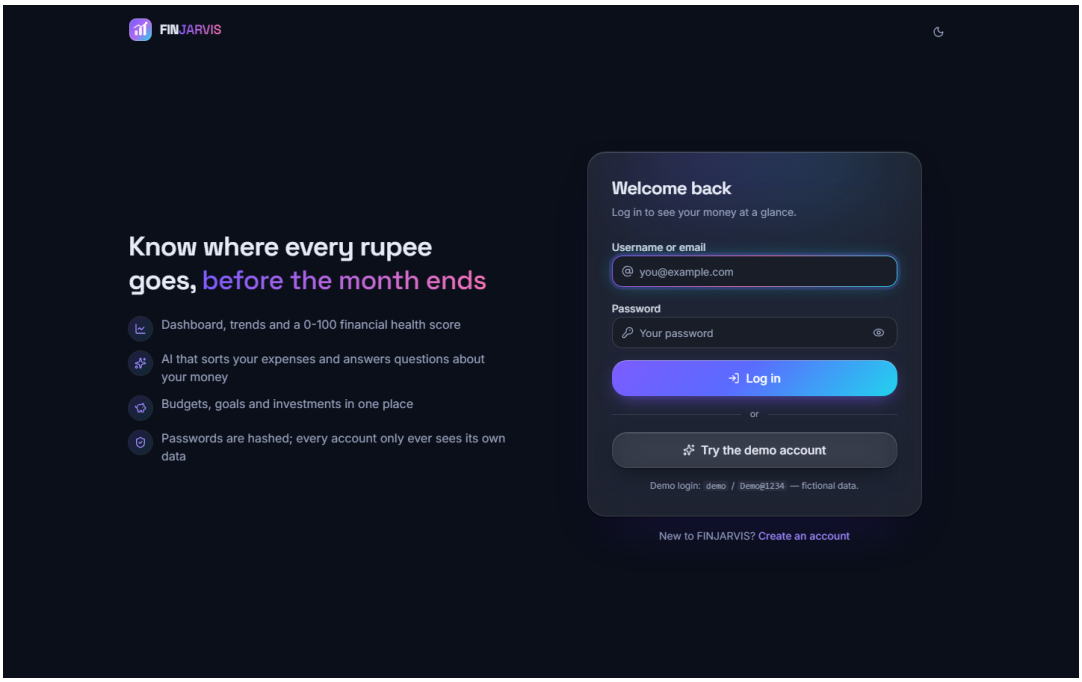


Figure 2: Login page: username or e-mail, password, a Try the demo account button and the demo credentials.

1.2.2 Dashboard

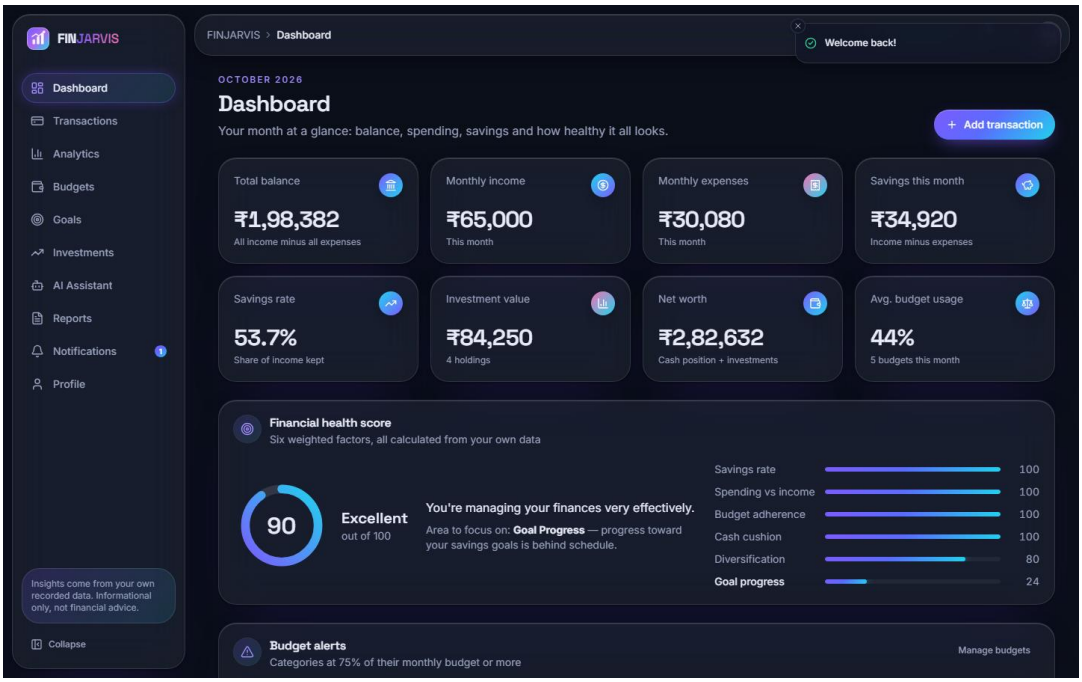


Figure 3: Dashboard for October 2026: balance INR 1,98,382, income INR 65,000, expenses INR 30,080, savings INR 34,920 (53.7 %), investments INR 84,250, net worth INR 2,82,632, average budget usage 44 %. The health card shows 90 (Excellent), component scores 100, 100, 100, 100, 80 and 24, and names Goal Progress as the area to focus on.



Figure 4: Dashboard charts: income against expenses, spending by category (led by Rent), the six-month trend and the allocation of the four holdings.

1.2.3 Transactions

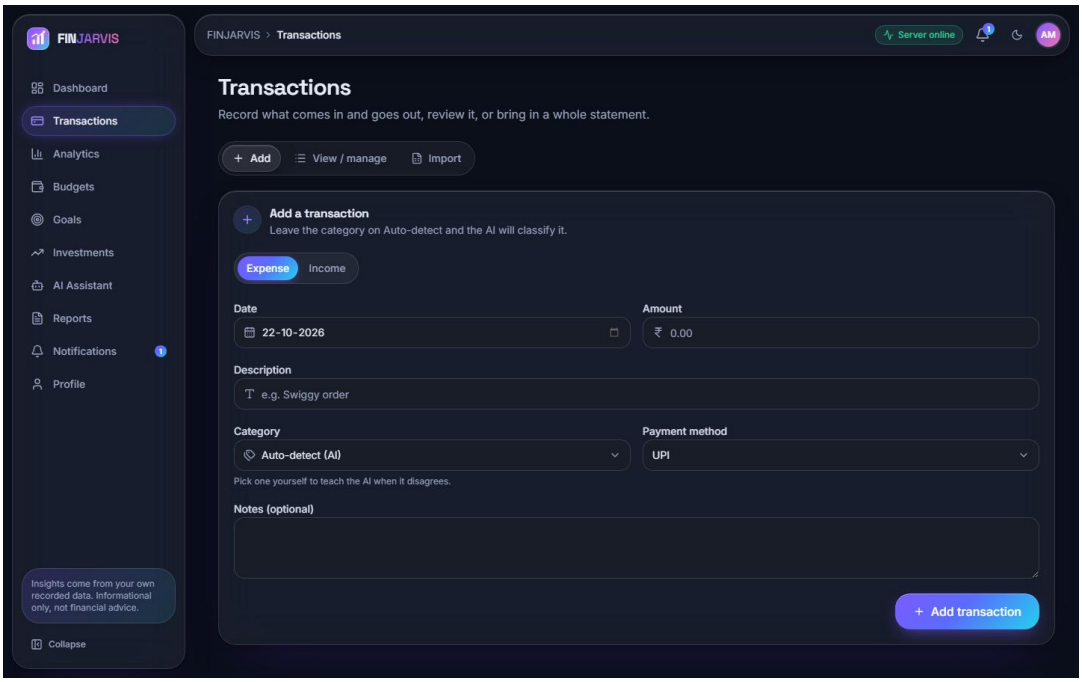


Figure 5: Add a transaction: type, date, amount, description, a category that defaults to Auto-detect (AI), payment method and notes; a manual category that differs teaches the model.

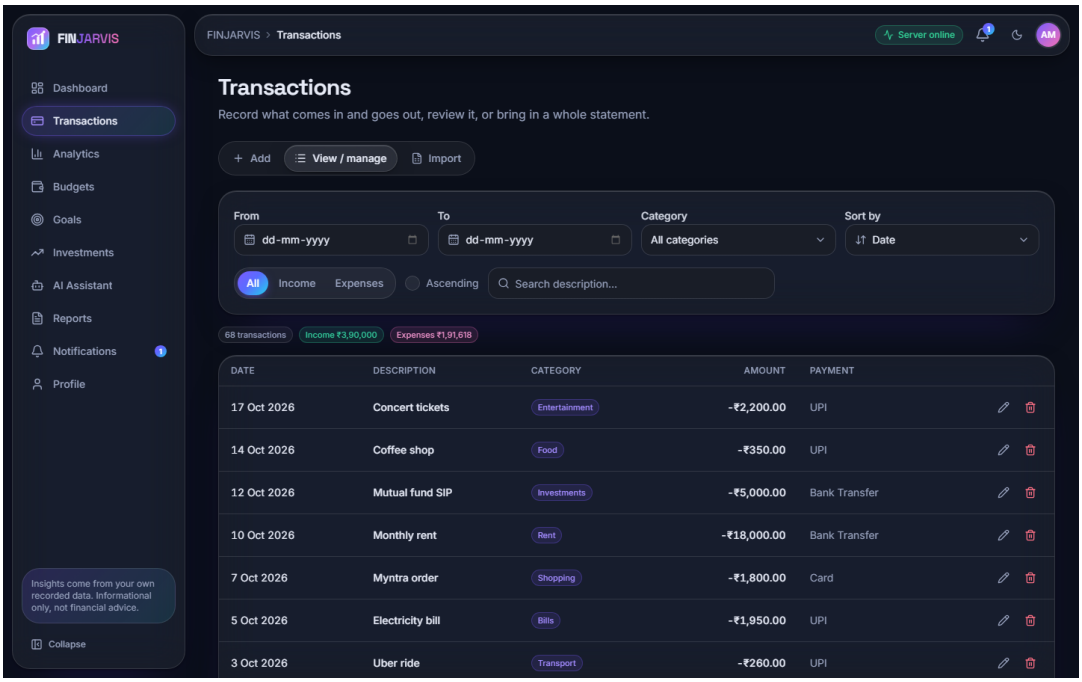


Figure 6: View and manage: 68 transactions, income INR 3,90,000 and expenses INR 1,91,618 for the six months, with filters, search, sort and edit and delete icons.

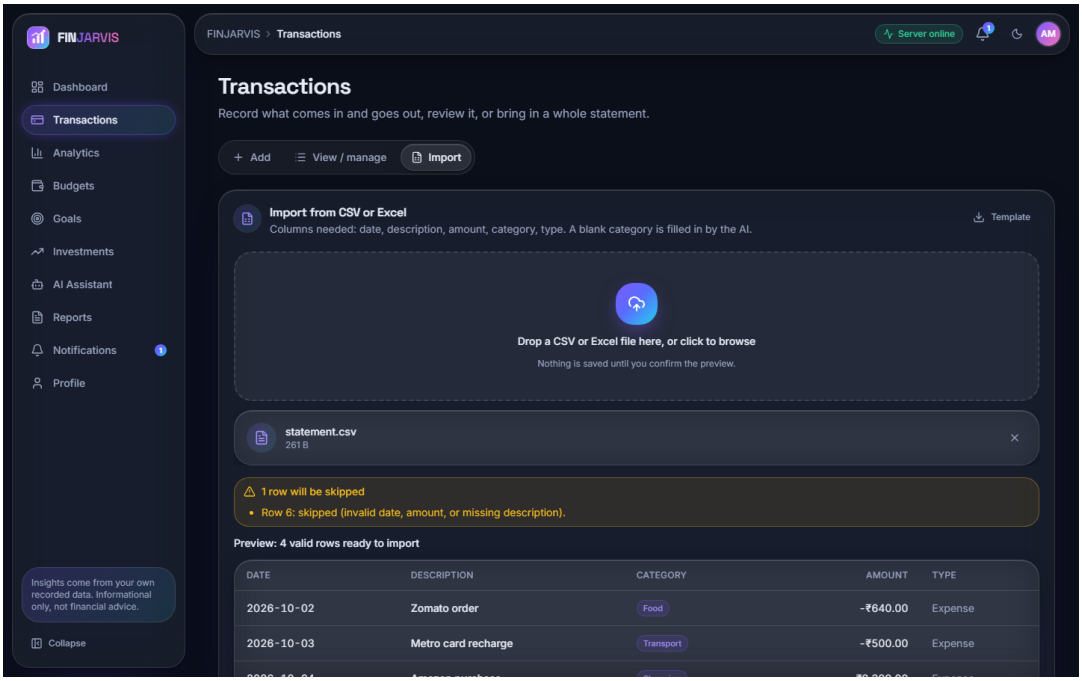


Figure 7: Import from CSV or Excel: the preview reports Row 6 as skipped and lists the four valid rows with categories the AI filled in; nothing is saved until the preview is confirmed.

1.2.4 Analytics and forecast

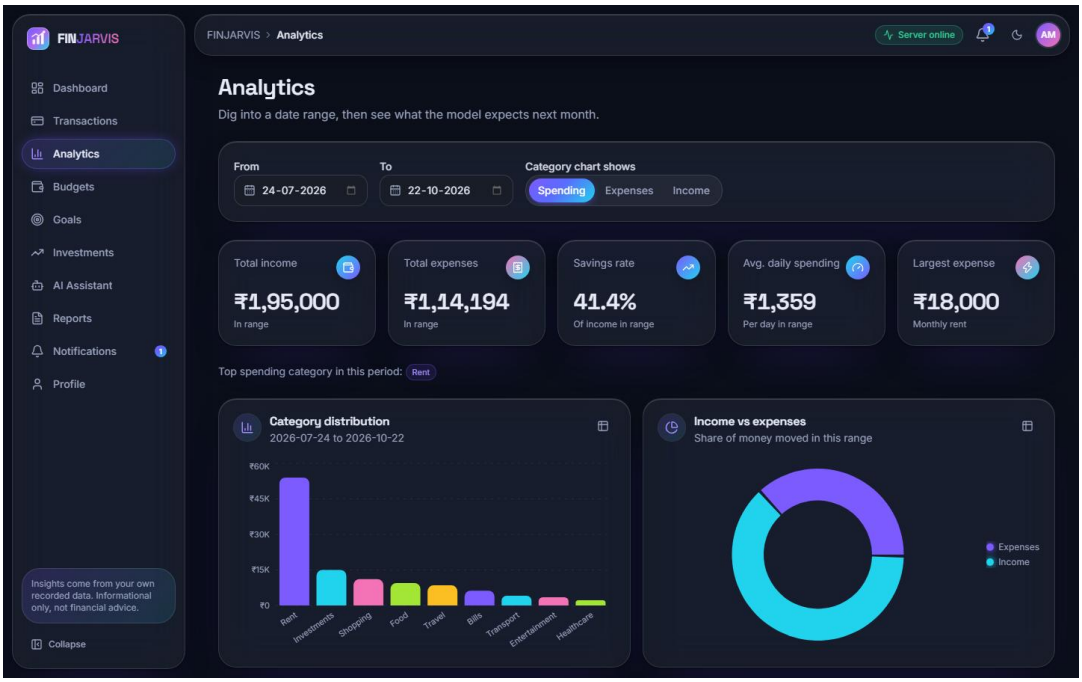


Figure 8: Analytics for 24 July to 22 October 2026: income INR 1,95,000, expenses INR 1,14,194, savings rate 41.4 %, daily average INR 1,359 and largest expense INR 18,000 (rent); the category chart is led by Rent, Investments and Shopping.



Figure 9: Spending trend and prediction: the expense curve for May to October 2026 and the prediction card: next month INR 39,471 from six months, recent three-month average INR 36,592, trend Increasing.

1.2.5 Budgets, goals and investments

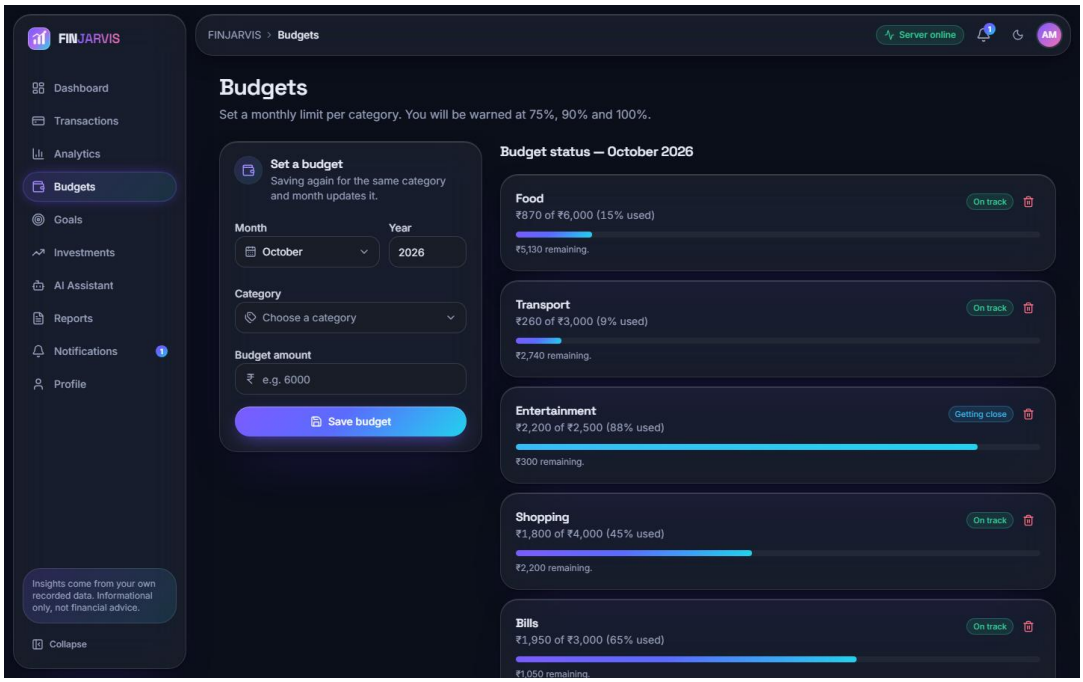


Figure 10: Budgets for October 2026: Food 15 %, Transport 9 %, Entertainment INR 2,200 of 2,500 (88 %, Getting close), Shopping 45 % and Bills 65 %, each with a progress bar; warnings are promised at 75, 90 and 100 %.

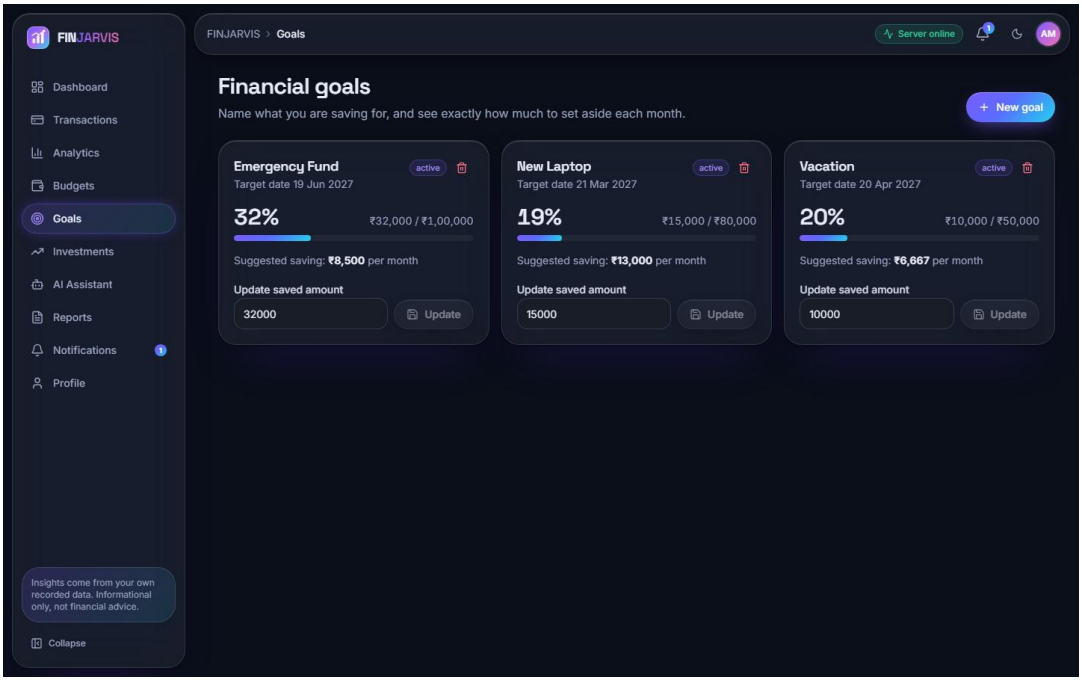


Figure 11: Financial goals: Emergency Fund 32 %, New Laptop 19 % and Vacation 20 %, each with the suggested monthly saving and an update field.

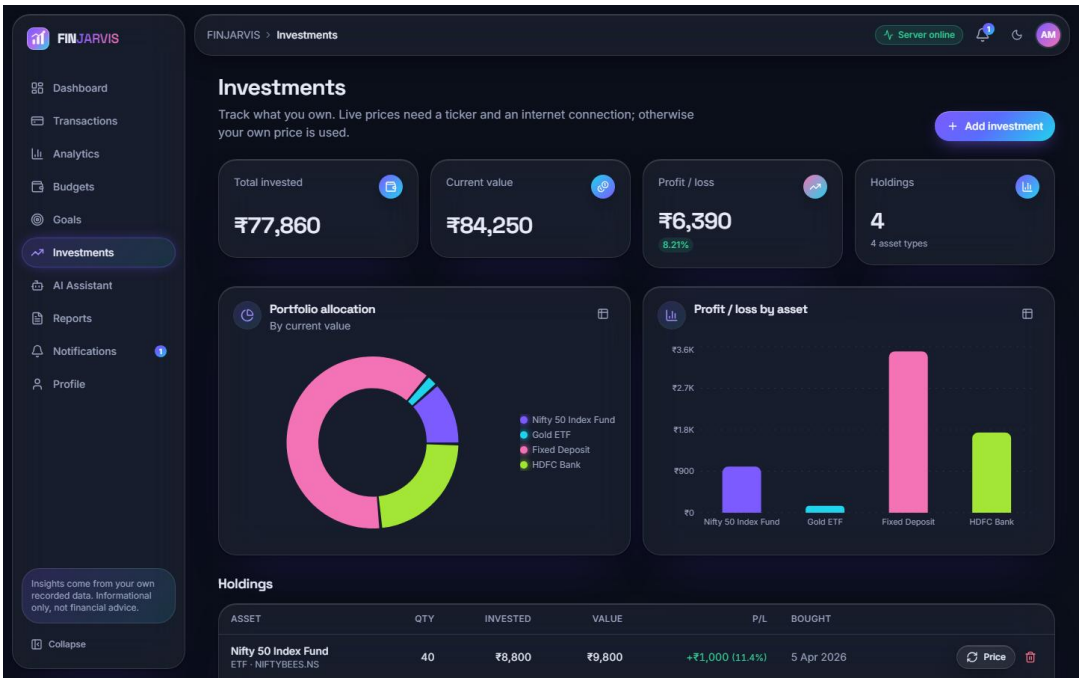


Figure 12: Investments: invested INR 77,860, value INR 84,250, profit INR 6,390 (8.21 %) over four holdings, with an allocation donut and profit and loss by asset.

1.2.6 Assistant, reports, notifications and profile

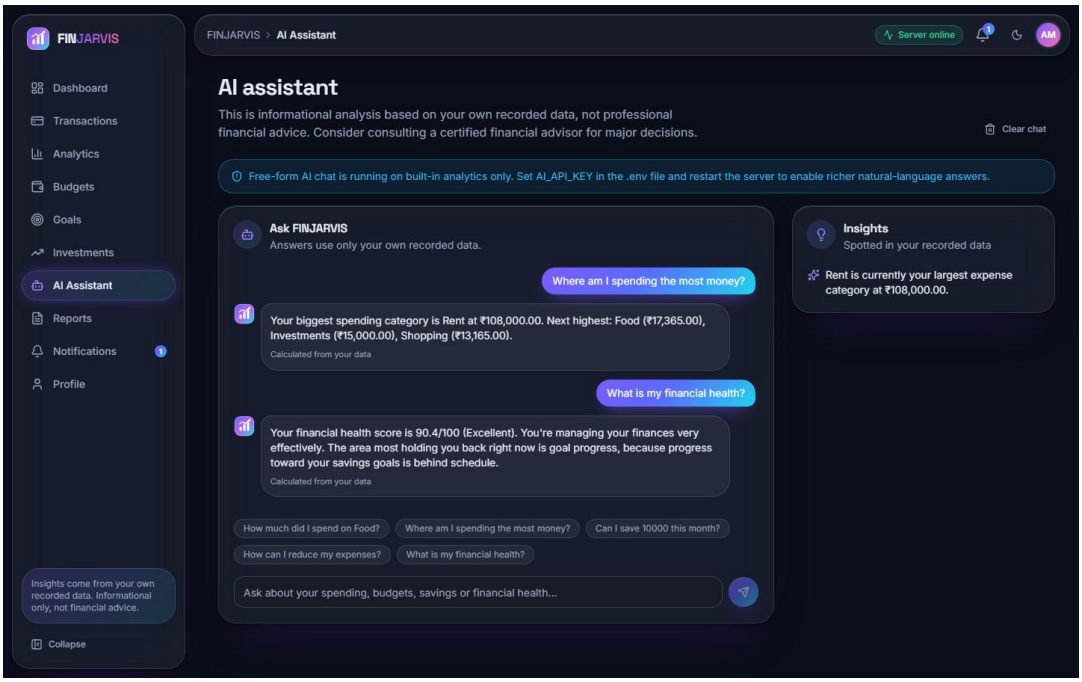


Figure 13: AI assistant: two questions answered from the user's own data (Rent at INR 108,000 is the top category; health 90.4, Excellent, weakest area goal progress). A banner notes that free-form answers are off because no AI_API_KEY is set.

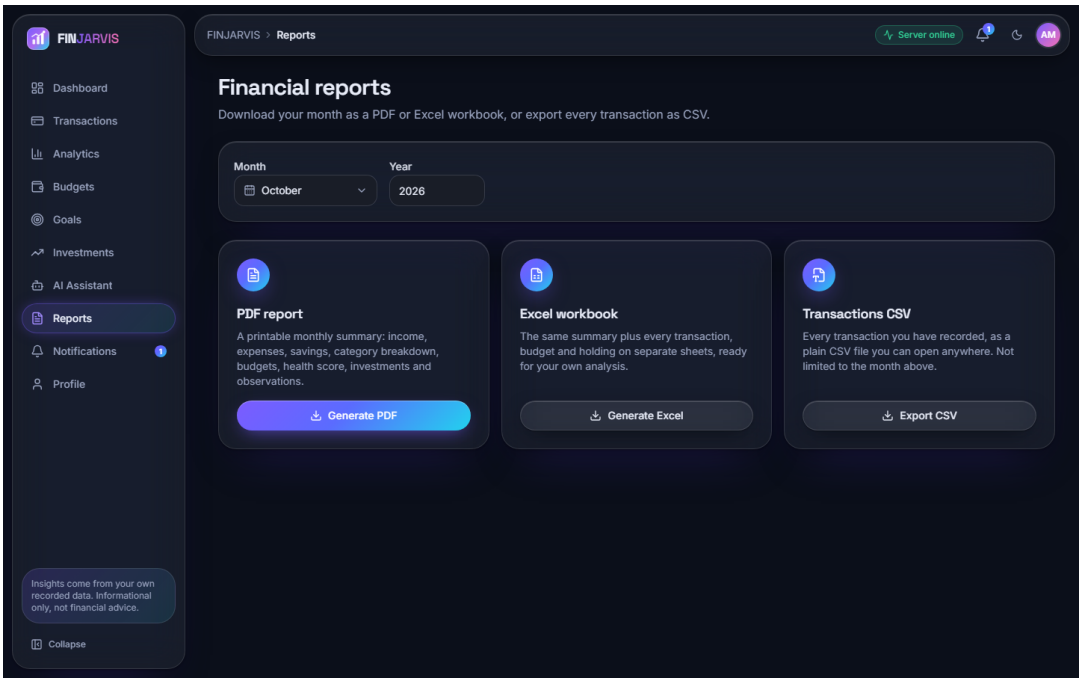


Figure 14: Financial reports: month and year selectors and three downloads, a PDF summary, an Excel workbook and a CSV of every transaction.

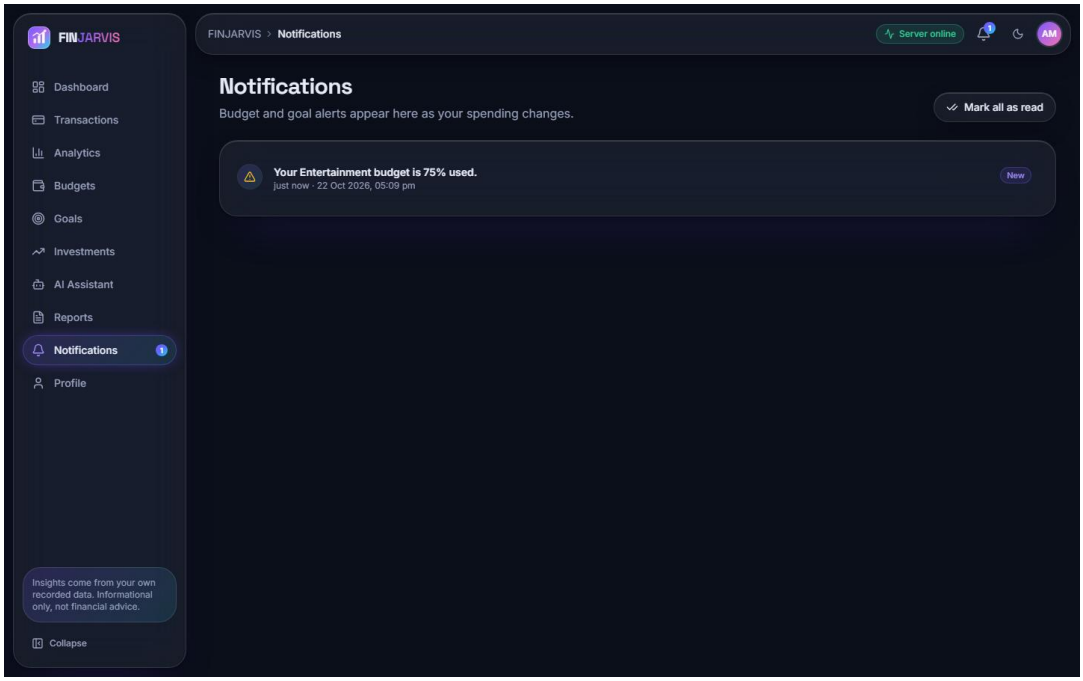


Figure 15: Notifications: one unread alert, Your Entertainment budget is 75% used, created when spending crossed the 75 % threshold.

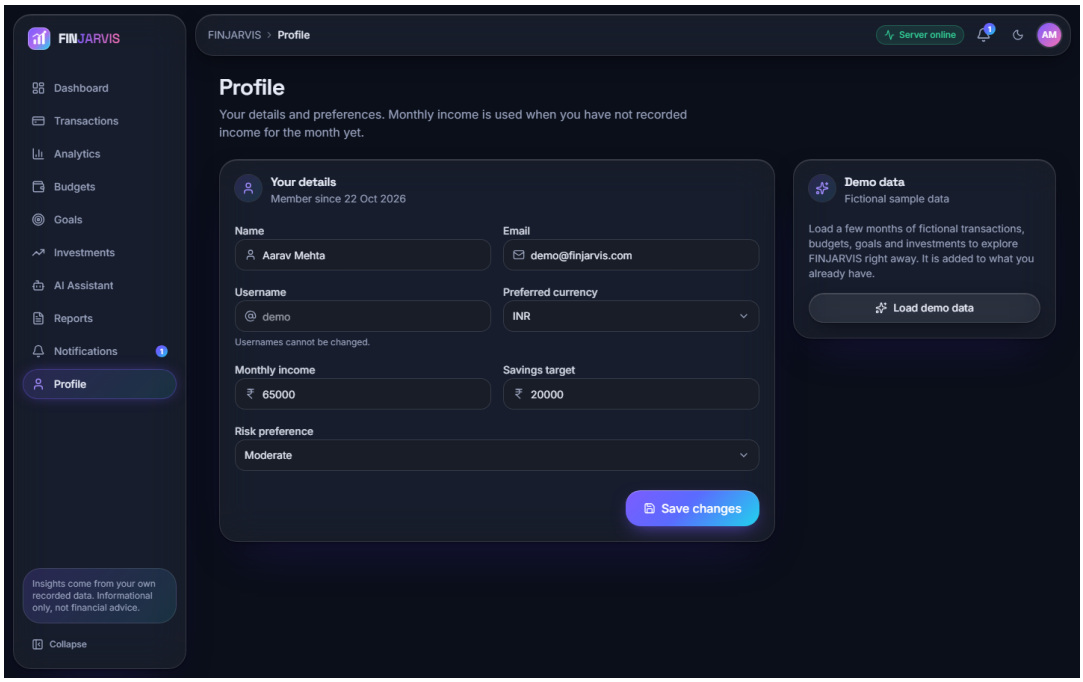


Figure 16: Profile: name, e-mail, currency INR, monthly income INR 65,000, savings target INR 20,000, risk preference Moderate and a card that loads the fictional demo data.

1.3 Expense categoriser

1.3.1 Accuracy on the probe set

The shipped model (TF-IDF with word unigrams and bigrams followed by logistic regression) labels 142 of the 198 probe descriptions correctly, an accuracy of **0.7172** (macro precision 0.8847, macro recall 0.7172, macro F1 **0.7614**). The headline hides a

sharp split between the tiers: 100.0 % on wording close to the seed list (66/66), 97.0 % on upper-case statement strings that contain a seed keyword (64/66), and only **18.2 %** on merchants it has never seen (12/66). On the same 198 descriptions the fallback keyword table scores 0.5152 (macro F1 0.6623) and always answering Food scores 0.0909; the learned model beats the table on every tier, but almost entirely in tiers A and B.

Table 3: Categoriser on the 198-description probe set.

Model	Accuracy	Macro P	Macro R	Macro F1	Tier A	Tier B	Tier C
TF-IDF + logistic regression (shipped)	0.7172	0.8847	0.7172	0.7614	1.000	0.970	0.182
Keyword table (fallback)	0.5152	0.9517	0.5152	0.6623	0.712	0.727	0.106
Majority class (Food)	0.0909	0.0083	0.0909	0.0152	0.091	0.091	0.091

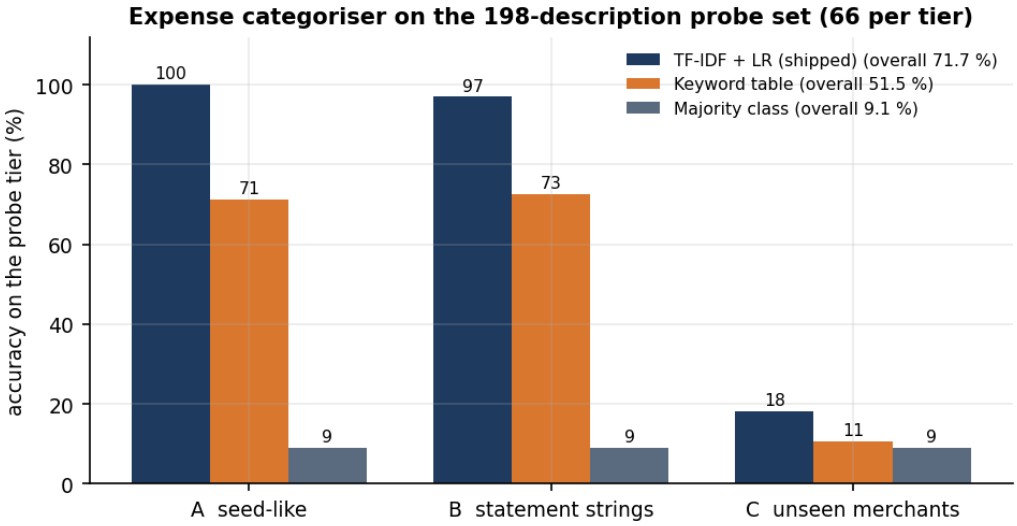


Figure 17: Accuracy of the three models on each probe tier; the shipped model falls from near 100 % to 18 % on unseen merchants.

On the 34-row bundled sample file the model labels all 34 rows as the file does (accuracy 1.000), but its vocabulary overlaps the seed list, so that figure is optimistic. Leave-one-out validation on the 51 seed pairs gives only **11.8 %** (6/51): each seed phrase has three to six siblings that hardly share words with it, so a held-out phrase cannot be recognised. This shows how small the seed list is and agrees with tier C.

1.3.2 Confusion matrix and per-category scores

The confusion matrix of the probe set is shown below. The diagonal holds 142 of 198 items. Almost all of the 56 errors fall in one column: **48 descriptions were labelled Transport**, 54 of the 56 errors belong to tier C and only 2 to tier B (`UPI-AMAZON-PAY-8821` labelled Entertainment, because the seed list has `amazon prime video`, and `UPI-HOSPITAL-BILL` labelled Bills). The cause is mechanical: a description made only of

unseen words gives an all-zero TF-IDF vector, so the decision comes from the intercepts alone and Transport has the largest: for the nonsense text `xyzabc123 qqq` the model returns Transport with probability 0.121, against 0.118 for Food and 0.118 for Entertainment. The model never outputs Other, so an unseen merchant is silently given a category.

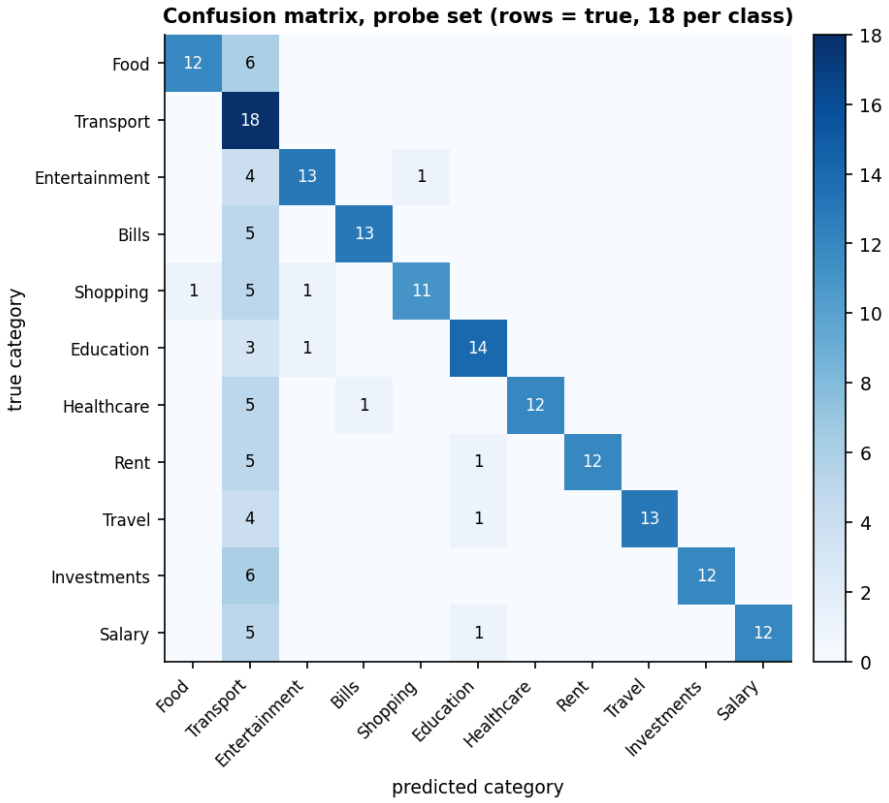


Figure 18: Confusion matrix on the probe set (rows are true categories, 18 descriptions each); the Transport column collects the unseen-merchant errors.

Table 4: Per-category scores on the probe set (18 descriptions per category).

Category	Precision	Recall	F1	Correct
Food	0.923	0.667	0.774	12/18
Transport	0.273	1.000	0.429	18/18
Entertainment	0.867	0.722	0.788	13/18
Bills	0.929	0.722	0.812	13/18
Shopping	0.917	0.611	0.733	11/18
Education	0.824	0.778	0.800	14/18
Healthcare	1.000	0.667	0.800	12/18
Rent	1.000	0.667	0.800	12/18
Travel	1.000	0.722	0.839	13/18
Investments	1.000	0.667	0.800	12/18
Salary	1.000	0.667	0.800	12/18

Precision is 1.000 for Healthcare, Rent, Travel, Investments and Salary; Transport's recall is 1.000 but its precision only **0.273**: it is the dump class.

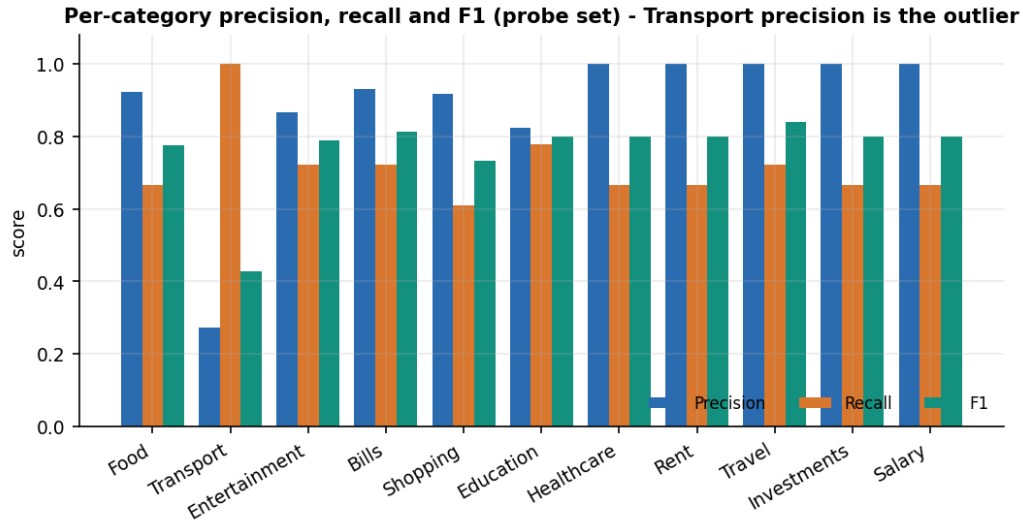


Figure 19: Precision, recall and F1 per category on the probe set; Transport recalls everything and is right on 27 % of what it claims.

1.3.3 Confidence and abstaining

The model's probabilities are low in absolute terms (eleven classes, 51 examples) but informative: the mean top probability is 0.197 when the label is right and 0.130 when it is wrong. Withholding predictions below 0.15 would still label 62.1 % of the items at 94.3 % accuracy; at 0.20 it would keep 32.3 % at 98.4 %, and at 0.25 7.6 % at 100 %. **The application applies no such threshold.** Asking the user when the model is unsure would turn the Transport dump into a prompt for a correction, the cheapest improvement available.

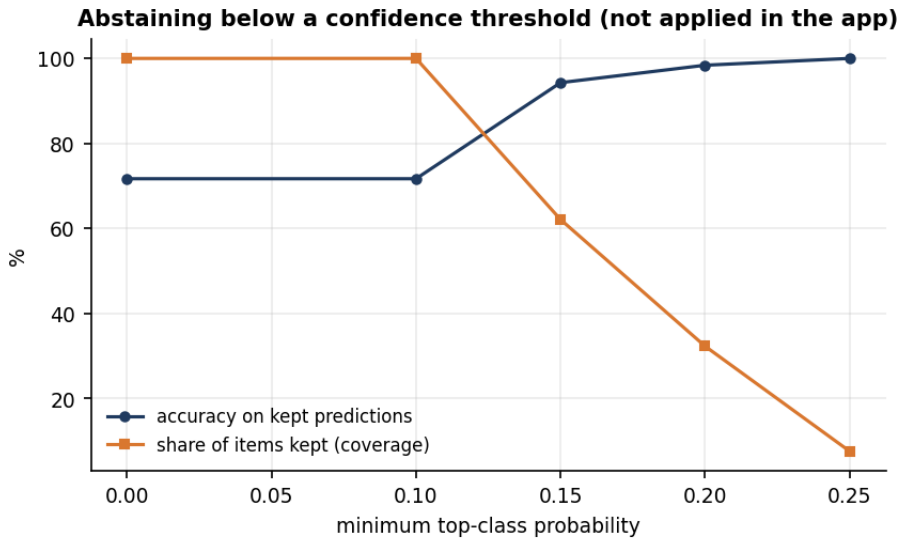


Figure 20: Accuracy on kept predictions and share of items kept as the minimum probability rises (computed offline; not part of the application).

1.3.4 Learning from corrections

When a user picks a category that differs from the prediction, the application retrains on the seed list, the user's history and the correction. Two experiments measure what that buys. In the first, k random tier-C items were added as corrections (30 draws per k) and accuracy was measured on the rest of tier C: it moved from 18.2 % with no corrections to 21.5 % with 48, so teaching the model some merchants hardly helps with others. In the second, one unseen merchant was corrected and then narrated three other ways (`UPI-X-4412`, `POS X`, `X 2`; 198 tests over 66 merchants): accuracy rose from 18.2 % to **100.0 %**. A correction fixes that merchant for good but does not generalise; the second test is easy by construction, because the variants share the corrected words.

Table 5: Tier-C accuracy after adding random corrections (mean and standard deviation over 30 draws).

Corrections added	Mean accuracy	Std
0	18.2 %	0.0
3	18.8 %	2.5
6	18.0 %	2.9
12	17.9 %	3.8
24	18.8 %	6.1
36	21.7 %	7.1
48	21.5 %	8.2

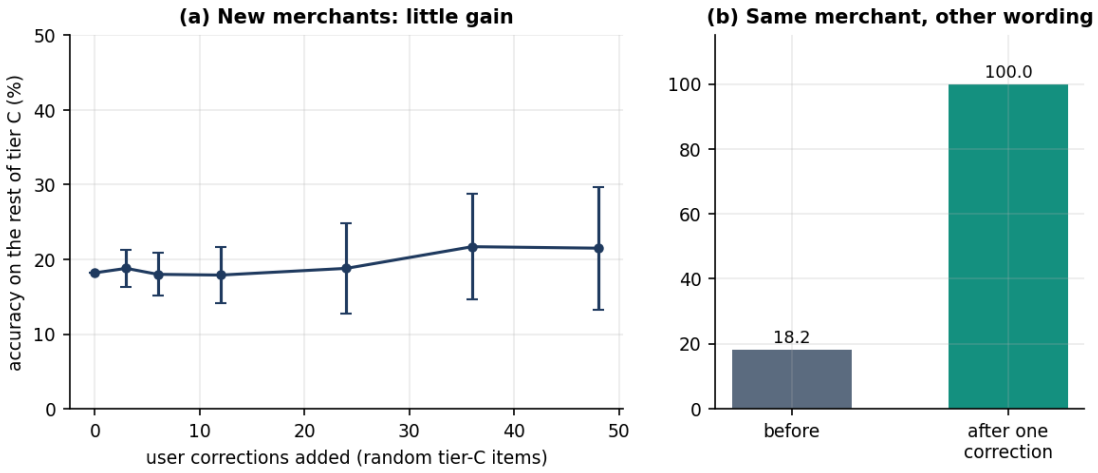


Figure 21: (a) Tier-C accuracy against the number of corrections; (b) accuracy on the same merchant written three other ways, before and after one correction.

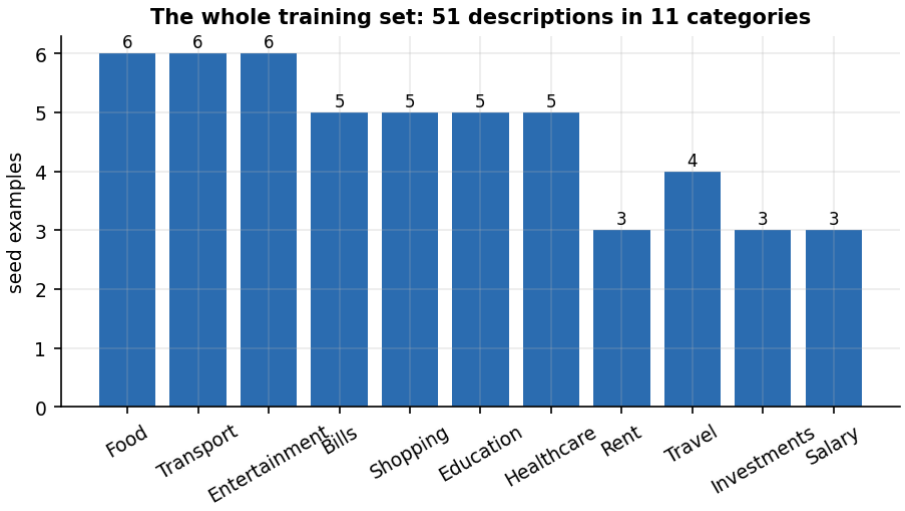


Figure 22: The complete training set: 51 descriptions, between 3 (Rent, Investments, Salary) and 6 (Food, Transport, Entertainment) per category.

One prediction takes 0.84 ms in process and retraining from the seed list 5.2 ms. The pickled model in `ml/` was trained under scikit-learn 1.5.0 and loads under 1.9.1 with an `InconsistentVersionWarning`; retraining removes it.

1.4 Spending forecast

1.4.1 Backtest on the demo account

The forecast fits a straight line to the monthly expense totals and extends it one month. It was tested by rolling origin on the demo account's six monthly totals (26,579, 26,878, 28,384, 36,118, 43,579, 30,080 INR for May to October 2026): months 4, 5 and 6 were each predicted from the months before them, so there are **only three test points**. The linear trend followed the August to September rise but overshot October, when spending fell from 43,579 to 30,080; its MAPE is **28.4 %** (MAE INR 9,597). The mean of all history was better (21.4 %, MAE INR 8,385), and so was the mean of the last three months (24.8 %). Three points cannot rank methods, but the trend model shows no demonstrated advantage here.

Table 6: One-step-ahead forecasts for August, September and October 2026 (INR) and their errors.

Method	Aug forecast	Sep forecast	Oct forecast	MAE	MAPE
Linear trend (app)	29,085	37,020	45,280	9,597	28.4 %
Mean of all history	27,280	29,490	32,308	8,385	21.4 %
Mean of last 3 months	27,280	30,460	36,027	9,301	24.8 %
Last value	28,384	36,118	43,579	9,565	27.8 %

The actual values were 36,118, 43,579, 30,080 INR. On the full six months the application predicts INR **39,471** for November (recent three-month average INR 36,592, trend increasing, 6 months used), as the Analytics page shows (Section 8.2.4). Per-category forecasts use the same fit: Rent 18,000, Investments 7,000, Travel 3,967, Shopping 3,314, Food 2,666, Bills 1,953, Entertainment 1,201, Healthcare 729, Transport 641 (INR); Rent equals its fixed INR 18,000, the one category where a straight line is exact.

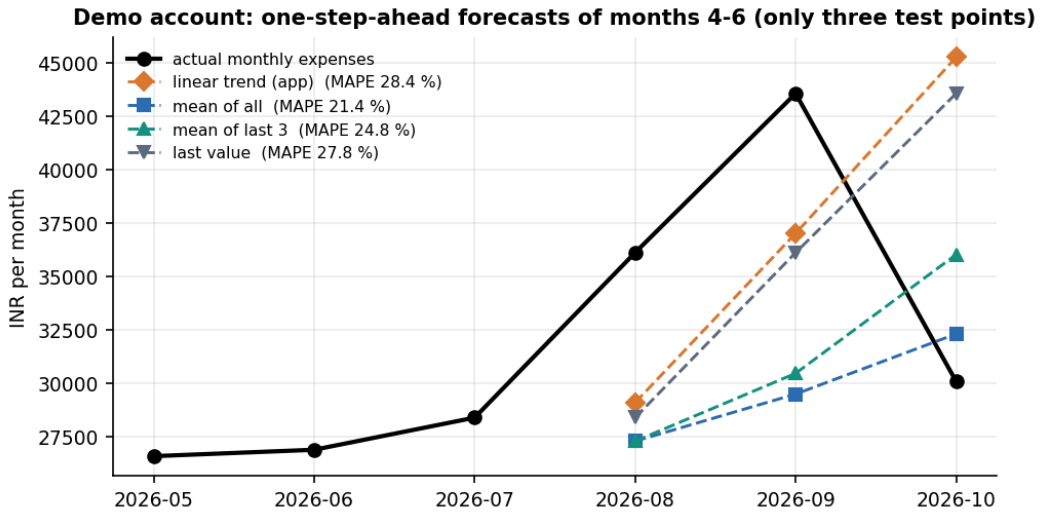


Figure 23: Actual monthly expenses against the four one-step-ahead forecasts for months 4 to 6 of the demo account.

1.4.2 Simulation: when does the trend help?

To learn more than three points can tell, the team simulated 2,000 users per setting (seed 7): a level drawn between 20,000 and 60,000, a linear drift of N(0, 3 % a month), Gaussian noise and a 10 % chance of a 50 % spike. **These are synthetic data and say nothing about real households**; they show how the methods behave when the generating process is known. At 5 % noise the trend gives a MAPE of 14.0 % against 13.1 % for the three-month mean; at 30 % noise 54.0 % and 51.1 %. The three-month mean was at least as good as the trend in every noise setting.

Table 7: Simulated one-step-ahead MAPE (%) against noise (12-month histories).

Noise (sd / level)	Linear trend (app)	Mean of all	Mean of last 3	Last value
0.05	13.99	16.77	13.11	13.19
0.10	18.62	20.72	17.07	18.84
0.20	30.84	31.06	28.23	32.44
0.30	53.99	54.26	51.11	56.11

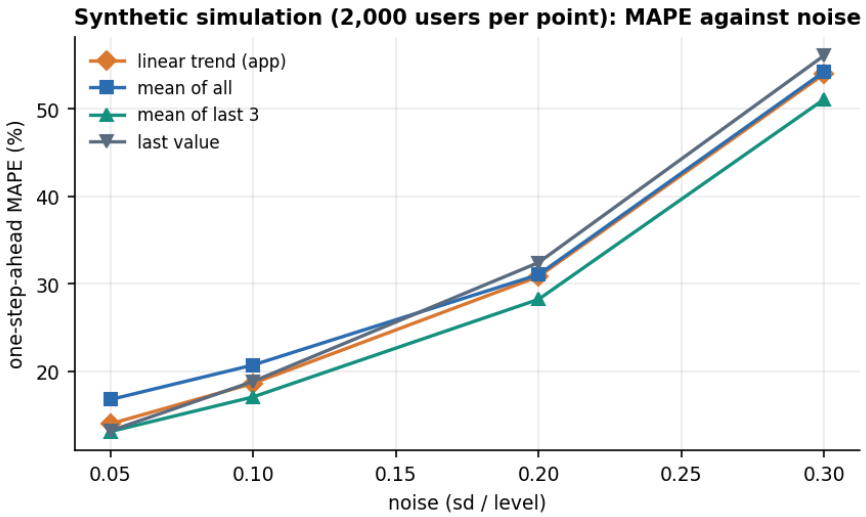


Figure 24: Simulated MAPE of the four methods against noise.

History length matters more than noise. At 10 % noise the trend needs about seven months before it overtakes a plain average: with 3 months its MAPE is 22.5 % against 14.9 % for the mean, with 7 months 16.9 % against 17.8 %, and with 11 months 17.5 % against 33.6 %, because the mean of all history ignores the drift. The three-month mean stays at least as good as the trend at every length (20.2 % at 11 months against 17.5 %). The application's minimum of 3 months is therefore too low: at that length the trend is the worst of the four.

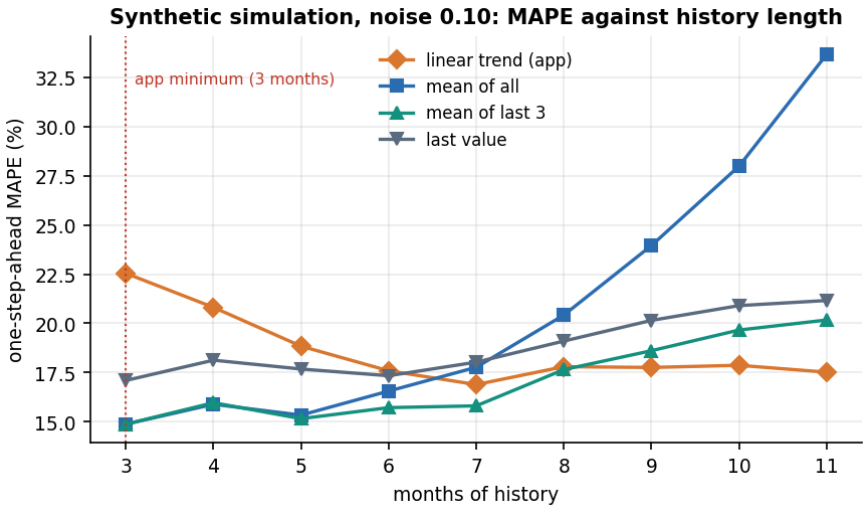


Figure 25: Simulated MAPE against months of history at 10 % noise; the dotted line marks the application's three-month minimum.

The base paper's LSTM reports RMSE 137.2, MAE 89.5 and R-squared 0.63 on a synthetic savings series of unstated units [1]; these cannot be compared with the errors above, since the series, target (savings, not expenses), units and validation differ.

1.5 Financial health score

The score is a weighted sum of six components, each scaled to 0 to 100 (Chapter 6). It has no training data, so the results are about its behaviour, not its accuracy. The demo account scores **90.4 out of 100, Excellent**; its weakest component is goal progress (24).

Table 8: Components of the demo account's score (October 2026).

Component	Weight	Component score	Contribution
Savings rate	0.30	100	30.0
Expense ratio	0.20	100	20.0
Budget adherence	0.15	100	15.0
Emergency fund	0.15	100	15.0
Investment diversification	0.10	80	8.0
Goal progress	0.10	24	2.4
Total	1.00		**90.4**

1.5.1 Sensitivity

From a baseline profile (income INR 65,000, expenses INR 32,500, a savings rate of 20 %, a balance of INR 100,000, two budgets at 60 % and 90 %, two asset types and goals at 40 %) the score is 80.7. Raising the savings rate from 0 % to 20 % lifts it from 50.7 to 80.7, and no further (the component saturates at 20 %). Raising monthly expenses from INR 20,000 to INR 75,000 lowers it from 85.5 to 56.3. A balance covering six months of expenses (INR 195,000 and above) lifts it to 88.0, where it stops. Goal progress from 0 % to 100 % adds 10.0 points and each extra asset type 2 points up to five types. The curves are monotone and saturate where the formulas cap.

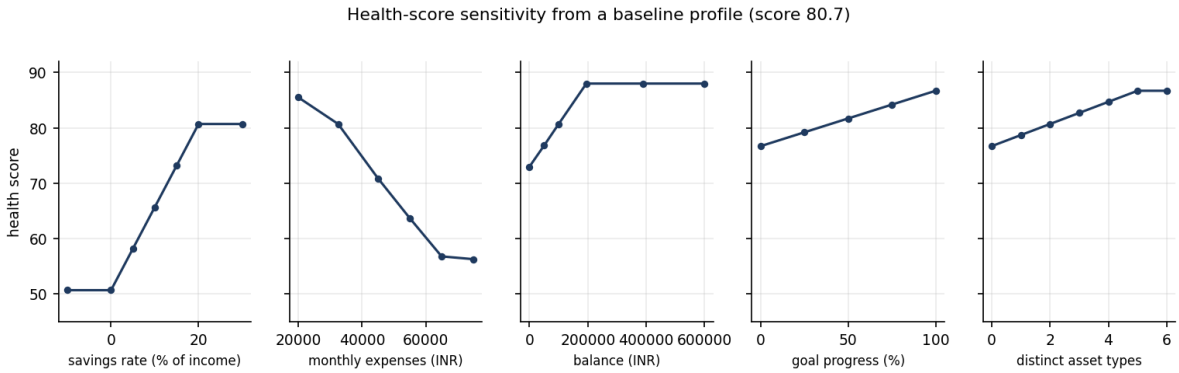


Figure 26: Health-score sensitivity to each input, changed one at a time from the baseline profile.

1.5.2 Behaviour over 5,000 random profiles

Over 5,000 random profiles the score ranged from 8.7 to 98.2 with a mean of 60.28; the ratings were Critical 1,425, Needs Improvement 680, Fair 1,083, Good 1,594 and Excellent 218. Raising the savings rate by five points never lowered the score (0

violations in 5,000 profiles). Two things limit what the score means. The savings rate and the expense ratio carry half of the weight and are strongly correlated (both derive from income minus expenses), so one habit is counted twice; and the weights are chosen by the team, not fitted to any outcome, with no user study. The score is a transparent summary, not a validated predictor of well-being.

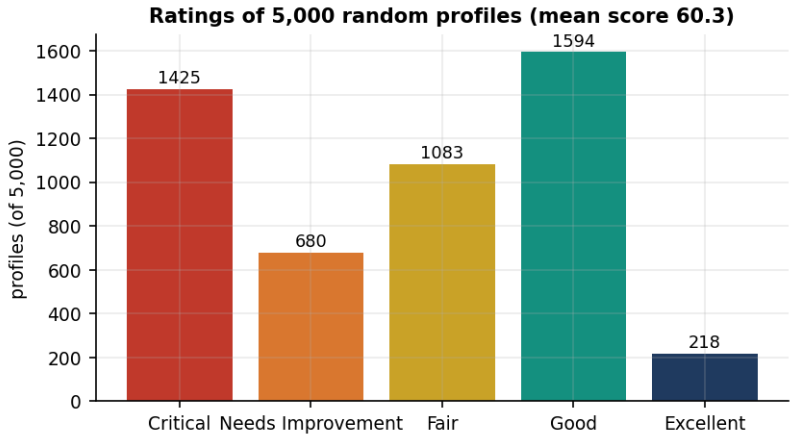


Figure 27: Ratings of 5,000 random profiles.

1.6 AI assistant

The assistant routes a question through five rule handlers (can I save, reduce expenses, financial health, where am I spending, spending on a named category) and, only if none matches and an `AI_API_KEY` is set, to an optional language model; otherwise it replies with a fixed help text. **The language-model layer was not exercised in any measurement.** The team wrote 26 questions, 20 answerable from the user's data and 6 off-topic, and asked them of the demo account. The rule layer answered **14 of 20 (70 %)** answerable questions and correctly declined 6 of 6 off-topic ones, wrongly answering 0. The 6 misses all fell to the fixed help text: "How much did I spend on groceries?"; "What is my biggest expense?"; "How much do I spend on eating out?"; "What was my largest purchase this month?"; "Will I run out of money this month?"; "Which category should I cut first?": phrasings the keyword routing does not recognise.

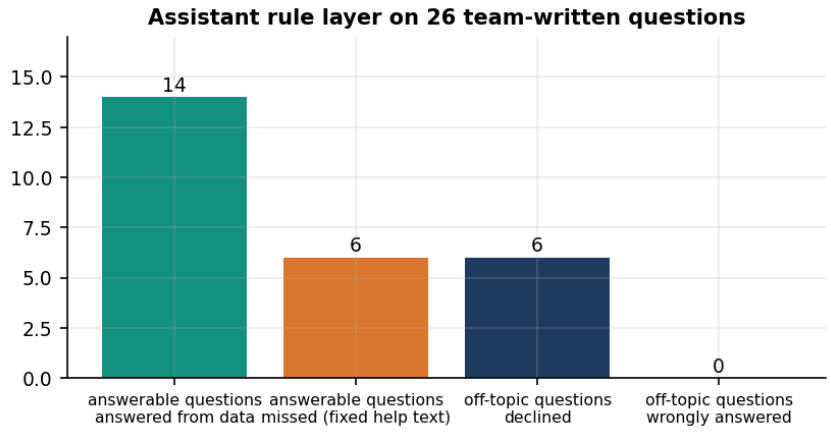


Figure 28: Outcome of the 26 test questions for the rule layer.

Table 9: Sample questions and the assistant's answers (demo account).

Question	Answered by	Answer (shortened)
How much did I spend on Food?	analytics	You've spent INR 17,365.00 on Food across your recorded transactions.
Can I save 5000 this month?	analytics	Based on this month so far, your income minus expenses leaves you INR 34,920.00, which comfortably covers a INR 5...
How can I reduce my expenses?	analytics	Your highest spending categories are: - Rent: INR 108,000.00 - Food: INR 17,365.00 - Investments: INR 15,000.00 A c...
What is my financial health?	analytics	Your financial health score is 90.4/100 (Excellent). You're managing your finances very effectively. The ar...
How much did I spend on groceries?	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...
What is my biggest expense?	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...
Tell me a joke	fixed help text	I can answer questions like "How much did I spend on Food?", "Where am I spending the most?", "Can I save INR ...

The numbers in the answers are correct: the Food answer is INR 17,365.00, equal to a direct SUM over the same records, and the top category is Rent at INR 108,000.00, as a group-by maximum gives. One quirk remains: Am I overspending on shopping? is answered with the Shopping total (INR 13,165), not a comparison with the budget.

1.7 Import pipeline, API latency and tests

1.7.1 Statement import

A 1,000-row CSV was prepared with all categories blank and 84 injected bad rows (28 unparsable dates, 17 negative amounts, 23 blank descriptions and 16 non-numeric amounts). The pipeline skipped exactly 84 rows, each with a Row n warning, and imported 916, auto-labelling all of them (916). Cleaning and categorising took 6.6 s and committing 26.04 s, **35.2 rows per second**: the commit opens one database session per row, the slow part and the first thing to change for large statements. The auto-labels

inherit the accuracy of Section 8.3; their correct share is unknown, as the generated file has no ground truth.

1.7.2 API latency

Reads are fast, with medians between 7.3 and 72.1 ms. Login takes 305 ms because bcrypt is deliberately slow; the heaviest calls are the Excel report (209 ms) and the 100-row import preview (667 ms). Single user, single process: no load or concurrency test was run.

Table 10: API latency in milliseconds (n = repetitions).

Endpoint	n	Median	p95	Max
login (bcrypt verify)	30	304.9	367.4	509.0
GET /health	30	4.3	19.0	21.3
GET /dashboard	30	67.2	129.3	194.6
GET /transactions	30	12.7	38.9	96.7
GET /analytics	30	72.1	128.5	170.0
GET /budgets	30	19.7	51.6	63.7
GET /goals	30	7.3	11.6	14.0
GET /investments	30	7.0	12.8	17.8
GET /assistant/insights	30	35.9	47.9	55.0
GET /notifications	30	18.1	33.2	49.3
GET /meta	30	7.2	11.8	12.8
POST /transactions/predict-category	30	7.9	20.5	30.6
POST /assistant/ask (analytics)	30	21.6	47.9	68.7
POST /assistant/ask (health)	30	28.4	47.8	86.4
POST /transactions (auto-categorise)	30	15.2	20.2	26.8
GET /reports/csv	10	17.7	30.5	59.1
GET /reports/excel	10	209.4	341.6	1165.2
GET /reports/pdf	10	135.5	201.8	211.9
POST /transactions/import/preview (100 rows)	10	667.0	933.3	941.0

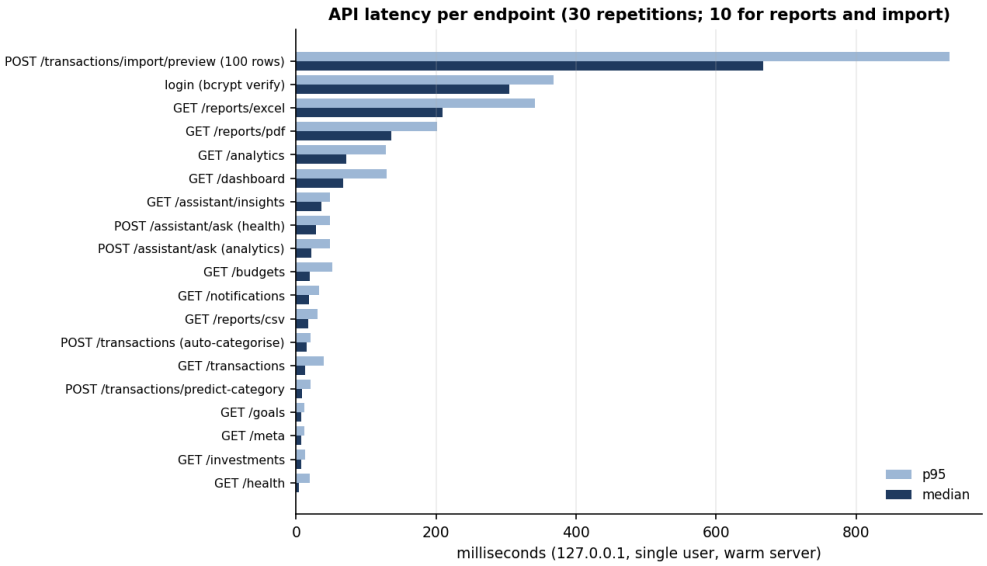


Figure 29: Median and 95th-percentile latency per endpoint.

1.7.3 Tests

`pytest` ran 38 tests and all 38 passed (authentication 9, budgets 6, financial health 6, machine learning 9, transactions 8) in 11.5 s, with 16 warnings from the scikit-learn pickle version. The end-to-end smoke script, which starts its own backend and a throw-away database, passed 49 of 49 checks (rejected weak passwords and tampered tokens, 401 on protected routes, per-user isolation, transactions, import, budgets, goals, investments, analytics, the assistant, the three downloads, profile and the demo-data loader). Not tested: browser-level UI tests (screens were captured by script and inspected by eye), concurrency, a security scan, accessibility, the language-model layer and the live price refresh.

1.8 Comparison with the base paper

The registered base paper [1] (Kharat et al.) builds four models in a Streamlit assistant: a BERT expense categoriser over 50 categories, a Q-learning budget recommender, an LSTM savings forecaster and an Isolation Forest anomaly detector. FINJARVIS implements simpler versions of the first and third modules, a rule-based substitute for the second and nothing for the fourth. The reported numbers belong to different data, label sets and models, so **they are shown side by side but must not be ranked**.

Table 11: Base paper against this work.

Module	Base paper [1]	FINJARVIS (this work)	Comparable?
Categorisation	BERT (bert-base-uncased, 10 epochs), 50 categories, SMOTE, 80/20 split of own data: accuracy 92.3 %, precision 90.8 %, recall 91.5 %, F1 91.1 %	TF-IDF + logistic regression, 11 categories, 51 training examples; team-written probe set: accuracy 71.7 %, macro F1 76.1 %; unseen merchants 18.2 %	No: different data, 50 against 11 labels, model size, split
Budget recommendation	Q-learning; convergence after about 150-200 of 300 episodes; no numeric accuracy	Budgets are set by the user; alerts at 75, 90 and 100 %; rule-based advice (top categories, trim 10-15 %)	No: nothing is learned here
Forecasting	LSTM on a synthetic savings series: RMSE 137.2, MAE 89.5, R-squared 0.63 (units not stated)	Linear trend on monthly expense totals: MAPE 28.4 % (MAE INR 9,597) on 3 backtest points; synthetic simulation in Section 8.4	No: different target, units and series
Anomaly detection	Isolation Forest: precision 93 %, recall 94 %, F1 94 %, on data with synthetic outliers	**Not implemented**; the nearest feature is the largest expense in a date range	Not applicable
Interface and delivery	Streamlit dashboard; Flask API mentioned	React 18 + FastAPI web app with 13 pages, 33 routes, SQLite; Streamlit original kept	Qualitative only

This work goes beyond the base paper in the product, not the models: a runnable multi-user web application with accounts, imports, budgets with alerts, goals, a portfolio, reports and an explainable score, plus a measurement of where its simple models fail. It falls short in modelling: no deep categoriser, learned budget policy, sequence forecaster

or anomaly detector. The batch abstract names Flask and MySQL; the shipped system uses FastAPI and SQLite, and the loan tracking in the abstract and periodic digest alerts were not built.

1.9 Sample inputs and outputs

The samples are taken from the stored experiment files; the demo account is fictional.

1.9.1 Forecast

```
# predict_next_month_expense(df) for the demo account's six months (Section 8.4)
{
  "available": true,
  "predicted_next_month_expense": 39470.53,
  "recent_average": 36592.33,
  "trend": "increasing",
  "months_used": 6
}
```

1.9.2 Categoriser

Table 12: Misclassified probe descriptions (first four tier-C errors and both tier-B errors).

Description	True category	Predicted	Tier
BigBasket delivery	Food	Transport	C
Dominos pizza	Food	Transport	C
Blinkit groceries	Food	Transport	C
Chaayos tea	Food	Transport	C
UPI-AMAZON-PAY-8821	Shopping	Entertainment	B
UPI-HOSPITAL-BILL	Healthcare	Bills	B

```
# text with no known word (all-zero feature vector)
input:     xyzabc123 qqq
predicted: Transport
probabilities (top 3): Transport 0.1210, Food 0.1183, Entertainment 0.1178
```

1.9.3 Assistant

```
Q: How much did I spend on Food?
A: You've spent INR 17,365.00 on Food across your recorded transactions.


Q: What is my financial health?
A: Your financial health score is 90.4/100 (Excellent). You're managing your finances very
effectively. The area most holding you back right now is goal progress,


Q: What is my biggest expense?   (not matched)
A: I can answer questions like "How much did I spend on Food?", "Where am I spending the most?",
"Can I save INR 5000 this month?", "...
```

1.10 Success and failure analysis

1.10.1 Where the product succeeds

- Every screen of Section 8.2 works end to end on the demo data: sign-up, login, transactions with import, budgets with alerts, goals, a portfolio, analytics, the assistant, three report formats and notifications.
- The rules are exact and tested: 38 unit tests and 49 smoke checks pass, the assistant's numbers equal direct SUMs, the import skips exactly the 84 bad rows, and the health score is monotone in the savings rate over 5,000 random profiles.
- The categoriser is dependable inside its vocabulary (100 % on seed-like wording, 97 % on statement strings with a known keyword), and one correction fixes a repeat merchant in all 198 test cases.
- Reads answer in under 72 ms at the median and a categorisation takes 0.84 ms.

1.10.2 Where it falls short

- **Unseen merchants are silently mislabelled.** 48 of the 56 probe errors are labelled Transport, tier-C accuracy is 18.2 % and the model never answers Other or asks.
- **Corrections do not generalise.** After 48 corrections tier-C accuracy is 21.5 %; only repeats of a corrected merchant are fixed.
- **The seed list is tiny** (51 pairs, 3 to 6 per class), so leave-one-out accuracy is 11.8 %.
- **The forecast is not better than an average with fewer than seven months of history**, and the three-month minimum lets it run in exactly that regime; the demo backtest has only three points.
- **The assistant misses natural phrasings** (6 of 20 answerable questions) and answers a budget question with a category total.
- **Anomaly detection, reinforcement-learning budgeting, a sequence forecaster and a language-model categoriser were not built**, so objective 4 is not met and objectives 3 and 5 only in simple form.
- **Import commits one row at a time** (35.2 rows per second).
- **Budget alerts are de-duplicated by message text**, so a budget that crosses 75 % again in a later month raises no new alert; the screenshot in Section 8.2.6 shows a

75 % message while Entertainment stands at 88 %, because the 90 % message appears only above 90 %.

- **Tokens are home-made signed tokens** (HMAC-SHA256, 7 days, no revocation) with a default development secret when `APP_SECRET_KEY` is unset, and the database is one SQLite file without migrations.

1.11 Limitations of the evaluation

- All application data are fictional; no real person's transactions or bank API were involved.
- The categoriser was scored on a 198-item probe set written by the team that wrote the seed list: a diagnostic, not a benchmark, whose tier proportions (one third unseen merchants) drive the headline accuracy.
- The forecast was backtested on three points of one fictional account; the simulation uses a generating process of the team's own choosing.
- The health-score weights are expert-chosen and not validated; no user study was run. The assistant was tested with 26 team-written questions and its language-model layer never ran.
- Latency was measured with one client over loopback on one machine; import speed shifts between runs when the machine is busy. Not exercised: live price refresh, concurrent users, mobile browsers, a security scan and accessibility testing.

1.12 Objectives versus outcomes

Table 13: The six project objectives against what was measured.

Objective	Status	Evidence
1. Secure user accounts with transaction import and manual entry	Achieved	Sign-up and login with bcrypt hashes, password rules and signed tokens; per-user data scoping; manual entry and CSV or Excel import with preview (916 of 1,000 rows imported, 84 bad rows skipped); 38 tests and 49 smoke checks pass
2. Categorise expenses automatically with text classification	Partly achieved	TF-IDF + logistic regression labels with 71.7 % probe accuracy and 76.1 % macro F1, but only 18.2 % on unseen merchants; 11 categories from 51 examples, no BERT
3. Forecast upcoming expenses and savings with time-series models	Partly achieved	A linear trend forecasts next month's expenses and per-category spend (INR 39,471 for November on the demo); MAPE 28.4 % on three points, no better than a mean under seven months; savings are not forecast directly; no ARIMA or LSTM
4. Detect anomalous transactions	Not achieved	No anomaly detector exists; only the largest expense in a date range is reported

Objective	Status	Evidence
5. Personalised budget recommendations and alerts on threshold breaches	Partly achieved	Budgets are set by the user; alerts fire at 75, 90 and 100 % of a budget; advice is rule-based (top categories, a 10 to 15 % trim) and uses the user's own income and spending; no learned policy
6. Visualise spending, category breakdown and forecasts on an interactive dashboard	Achieved	React dashboard, analytics, budgets, goals and investment pages with charts and a forecast card (Section 8.2); health score with six explained components; reports in PDF, Excel and CSV

1.13 Summary of findings

- FINJARVIS runs as a complete personal finance web application (13 pages, 33 routes, 6 tables, 38 tests and 49 smoke checks passing); reads answer in under 72 ms at the median.
- The categoriser works on familiar wording (100 % and 97 % on tiers A and B) but fails on unseen merchants (18.2 %), mostly by answering Transport; a confidence threshold of 0.15 would keep 62 % of items at 94 % accuracy but is not applied.
- A correction fixes a repeated merchant completely, but 48 corrections lift accuracy on new merchants only from 18 % to 22 %.
- The linear-trend forecast overshot October on the demo account (MAPE 28.4 % against 21.4 % for a plain mean) and, in simulation, needs about seven months of history to beat an average; the three-month minimum is too low.
- The health score is explainable and monotone (demo account 90.4, Excellent); its weights are not validated. The assistant answers 14 of 20 answerable questions from the user's data and declines all off-topic ones; its language-model layer is untested.
- Anomaly detection, reinforcement-learning budgeting, an LSTM forecaster, a BERT categoriser, loan tracking and scheduled digests were not built; the base paper's four-model design is matched only partly, and its reported numbers are not comparable with these.
- Objectives 1 and 6 are met; objectives 2, 3 and 5 are met in simple form; objective 4 is not met.